

RIDELINK DISTRIBUTED MICROSERVICES SYSTEM

Application Development (AD) Final Group Project Report

GitHub Repository URL:

<https://github.com/IT24104304/RideLink.git>

Group Role	Member Name	Student ID	Primary Service Module
Group Leader	Kulasuriya W.N.N	IT24104304	Driver & Vehicle Service
Member 2	Perera H.G.K.D	IT24104143	Ride Management Service
Member 3	Bandara J.K.T.I	IT24104276	Account Service
Member 4	Karunarathna H.W.D.S.S	IT24104353	Fare & Payment Service

Executive Summary:

RideLink is a comprehensive, production-grade distributed microservices application designed for urban transportation management. Built using Spring Boot, Java 21, and MongoDB, the system comprises four decoupled microservices: Account Service, Driver & Vehicle Service, Ride Management Service, and Fare & Payment Service. This report outlines the system architecture, REST API design, domain models, and complete source code for all components.

1. System Architecture & Component Design

RideLink adopts a modern microservice-oriented architecture. Each service operates independently with its own dedicated data repository (MongoDB) ensuring low coupling and high reliability. Communication occurs via standardized RESTful APIs returning structured JSON object representations.

Service Module	Port	Database Name	Primary Responsibilities
Account Service	8080	ridelink_account	User authentication, profile management, role assignment
Ride Management Service	8081	ridelink_ride	Ride request lifecycle, booking status, driver assignment
Driver & Vehicle Service	8082	ridelink_driver_vehicle	Driver profile, license tracking, vehicle registration, location & availability
Fare & Payment Service	8083	ridelink_fare_payment	Fare calculation, fare estimation, payment execution (Cash/Card)

2. Driver & Vehicle Service Module

Assigned Lead: Kulasuriya W.N.N (Student ID: IT24104304 - Group Leader)

Overview: Manages driver profiles, license numbers, location coordinates, availability statuses (AVAILABLE, UNAVAILABLE, OFFLINE), vehicle registration, and status lifecycle.

■ File: driver-vehicle-service/.mvn\wrapper\maven-wrapper.properties

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.16/apache-maven-3.9.16-bin.zip
```

■ File: driver-vehicle-service/src\main\resources\application.properties

```
spring.application.name=driver-vehicle-service

spring.data.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/ridelink_driver_vehicle}
spring.data.mongodb.database=ridelink_driver_vehicle

server.port=8082
```

■ File: driver-vehicle-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.ridelink</groupId>
  <artifactId>driver-vehicle-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name/>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.8.5</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
```

```
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
    </plugins>
</build>

</project>
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/DriverVehicleServiceApplication.java

```
package com.ridelink.driver_vehicle_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DriverVehicleServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(DriverVehicleServiceApplication.class, args);
    }

}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/config/OpenApiConfig.java

```
package com.ridelink.driver_vehicle_service.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI driverVehicleServiceOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Driver & Vehicle Service API")
                .description("REST API documentation for RideLink Driver and Vehicle Microservice")
                .version("1.0.0"));
    }

}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/controller/DriverController.java

```
package com.ridelink.driver_vehicle_service.controller;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.service.DriverService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/drivers")
public class DriverController {

    private final DriverService driverService;

    public DriverController(DriverService driverService) {
        this.driverService = driverService;
    }

    @PostMapping
    public ResponseEntity<DriverResponse> createDriver(@Valid @RequestBody CreateDriverRequest request) {
        DriverResponse response = driverService.createDriver(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/{driverId}")
    public ResponseEntity<DriverResponse> getDriverById(@PathVariable String driverId) {
        DriverResponse response = driverService.getDriverById(driverId);
        return ResponseEntity.ok(response);
    }

    @PutMapping("/{driverId}")
    public ResponseEntity<DriverResponse> updateDriver(
        @PathVariable String driverId,
        @Valid @RequestBody UpdateDriverRequest request) {
        DriverResponse response = driverService.updateDriver(driverId, request);
        return ResponseEntity.ok(response);
    }
}
```

```
@PatchMapping("/{driverId}/availability")
public ResponseEntity<DriverResponse> updateAvailability(
    @PathVariable String driverId,
    @Valid @RequestBody AvailabilityUpdateRequest request) {
    DriverResponse response = driverService.updateAvailability(driverId, request);
    return ResponseEntity.ok(response);
}

@PatchMapping("/{driverId}/location")
public ResponseEntity<DriverResponse> updateLocation(
    @PathVariable String driverId,
    @Valid @RequestBody LocationUpdateRequest request) {
    DriverResponse response = driverService.updateLocation(driverId, request);
    return ResponseEntity.ok(response);
}

@PatchMapping("/{driverId}/service-area")
public ResponseEntity<DriverResponse> updateServiceArea(
    @PathVariable String driverId,
    @Valid @RequestBody ServiceAreaUpdateRequest request) {
    DriverResponse response = driverService.updateServiceArea(driverId, request);
    return ResponseEntity.ok(response);
}

@GetMapping("/available")
public ResponseEntity<List<DriverResponse>> getAvailableDrivers(
    @RequestParam(required = false) String serviceArea) {
    List<DriverResponse> responses;
    if (serviceArea != null && !serviceArea.isBlank()) {
        responses = driverService.getAvailableDriversByServiceArea(serviceArea);
    } else {
        responses = driverService.getAvailableDrivers();
    }
    return ResponseEntity.ok(responses);
}

@DeleteMapping("/{driverId}")
public ResponseEntity<Void> deleteDriver(@PathVariable String driverId) {
    driverService.deleteDriver(driverId);
```

```
    return ResponseEntity.noContent().build();
}
```

```
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/controller/VehicleController.java

```
package com.ridelink.driver_vehicle_service.controller;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.service.VehicleService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
public class VehicleController {

    private final VehicleService vehicleService;

    public VehicleController(VehicleService vehicleService) {
        this.vehicleService = vehicleService;
    }

    @PostMapping("/api/vehicles")
    public ResponseEntity<VehicleResponse> createVehicle(@Valid @RequestBody CreateVehicleRequest request) {
        VehicleResponse response = vehicleService.createVehicle(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/api/vehicles/{vehicleId}")
    public ResponseEntity<VehicleResponse> getVehicleById(@PathVariable String vehicleId) {
        VehicleResponse response = vehicleService.getVehicleById(vehicleId);
        return ResponseEntity.ok(response);
    }

    @PutMapping("/api/vehicles/{vehicleId}")
    public ResponseEntity<VehicleResponse> updateVehicle(
        @PathVariable String vehicleId,
        @Valid @RequestBody UpdateVehicleRequest request) {
        VehicleResponse response = vehicleService.updateVehicle(vehicleId, request);
        return ResponseEntity.ok(response);
    }

    @PatchMapping("/api/vehicles/{vehicleId}/status")
    public ResponseEntity<VehicleResponse> updateVehicleStatus(
        @PathVariable String vehicleId,
        @Valid @RequestBody VehicleStatusUpdateRequest request) {
        VehicleResponse response = vehicleService.updateVehicleStatus(vehicleId, request);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/api/drivers/{driverId}/vehicles")
    public ResponseEntity<List<VehicleResponse>> getVehiclesByDriverId(@PathVariable String driverId) {
        List<VehicleResponse> responses = vehicleService.getVehiclesByDriverId(driverId);
        return ResponseEntity.ok(responses);
    }

    @DeleteMapping("/api/vehicles/{vehicleId}")
    public ResponseEntity<Void> deleteVehicle(@PathVariable String vehicleId) {
        vehicleService.deleteVehicle(vehicleId);
        return ResponseEntity.noContent().build();
    }
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/AvailabilityUpdateRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.DriverStatus;
import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class AvailabilityUpdateRequest {

    @NotNull(message = "Availability status is required")
    private DriverStatus availabilityStatus;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/CreateDriverRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class CreateDriverRequest {

    @NotBlank(message = "Account ID is required")
    private String accountId;

    @NotBlank(message = "License number is required")
    private String licenseNumber;

    @NotBlank(message = "Service area is required")
    private String serviceArea;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/CreateVehicleRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.VehicleType;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class CreateVehicleRequest {

    @NotBlank(message = "Driver ID is required")
    private String driverId;

    @NotBlank(message = "Registration number is required")
    private String registrationNumber;

    @NotBlank(message = "Brand is required")
    private String brand;

    @NotBlank(message = "Model is required")
    private String model;

    @NotNull(message = "Vehicle type is required")
    private VehicleType vehicleType;

    private String color;

    @NotNull(message = "Year is required")
    private Integer year;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/DriverResponse.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.DriverStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDateTime;

@Getter
@Setter
@Builder
@AllArgsConstructor
public class DriverResponse {

    private String id;

    private String accountId;

    private String licenseNumber;

    private String serviceArea;

    private DriverStatus availabilityStatus;

    private Double latitude;

    private Double longitude;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/LocationUpdateRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class LocationUpdateRequest {

    @NotNull(message = "Latitude is required")
    private Double latitude;

    @NotNull(message = "Longitude is required")
    private Double longitude;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/ServiceAreaUpdateRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class ServiceAreaUpdateRequest {

    @NotBlank(message = "Service area is required")
    private String serviceArea;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/UpdateDriverRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class UpdateDriverRequest {

    private String licenseNumber;

    private String serviceArea;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/UpdateVehicleRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.VehicleType;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class UpdateVehicleRequest {

    private String registrationNumber;

    private String brand;

    private String model;

    private VehicleType vehicleType;

    private String color;

    private Integer year;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/VehicleResponse.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.VehicleStatus;
import com.ridelink.driver_vehicle_service.model.VehicleType;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDateTime;

@Getter
@Setter
@Builder
@AllArgsConstructor
public class VehicleResponse {

    private String id;

    private String driverId;

    private String registrationNumber;

    private String brand;

    private String model;

    private VehicleType vehicleType;

    private String color;

    private Integer year;

    private VehicleStatus status;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/dto/VehicleStatusUpdateRequest.java

```
package com.ridelink.driver_vehicle_service.dto;

import com.ridelink.driver_vehicle_service.model.VehicleStatus;
import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class VehicleStatusUpdateRequest {

    @NotNull(message = "Vehicle status is required")
    private VehicleStatus status;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/exception/DriverNotFoundException.java

```
package com.ridelink.driver_vehicle_service.exception;

public class DriverNotFoundException extends RuntimeException {

    public DriverNotFoundException(String id) {
        super("Driver not found with id: " + id);
    }
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/exception/DuplicateLicenseException.java

```
package com.ridelink.driver_vehicle_service.exception;

public class DuplicateLicenseException extends RuntimeException {

    public DuplicateLicenseException(String licenseNumber) {
        super("Driver with license number " + licenseNumber + " already exists");
    }
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/exception/DuplicateVehicleRegistrationException.java

```
package com.ridelink.driver_vehicle_service.exception;

public class DuplicateVehicleRegistrationException extends RuntimeException {

    public DuplicateVehicleRegistrationException(String registrationNumber) {
        super("Vehicle with registration number " + registrationNumber + " already exists");
    }
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.driver_vehicle_service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(DriverNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleDriverNotFound(
        DriverNotFoundException ex) {

        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.NOT_FOUND.value());
        response.put("error", "Driver Not Found");
        response.put("message", ex.getMessage());

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(response);
    }

    @ExceptionHandler(DuplicateLicenseException.class)
    public ResponseEntity<Map<String, Object>> handleDuplicateLicense(
        DuplicateLicenseException ex) {

        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.CONFLICT.value());
        response.put("error", "Duplicate License Number");
        response.put("message", ex.getMessage());
    }
}
```

```

        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(response);
    }

    @ExceptionHandler(VehicleNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleVehicleNotFound(
        VehicleNotFoundException ex) {

        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.NOT_FOUND.value());
        response.put("error", "Vehicle Not Found");
        response.put("message", ex.getMessage());

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(response);
    }

    @ExceptionHandler(DuplicateVehicleRegistrationException.class)
    public ResponseEntity<Map<String, Object>> handleDuplicateVehicleRegistration(
        DuplicateVehicleRegistrationException ex) {

        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.CONFLICT.value());
        response.put("error", "Duplicate Vehicle Registration");
        response.put("message", ex.getMessage());

        return ResponseEntity
            .status(HttpStatus.CONFLICT)
            .body(response);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationErrors(
        MethodArgumentNotValidException ex) {

        Map<String, Object> errors = new HashMap<>();

```

```

        ex.getBindingResult()
            .getFieldErrors()
            .forEach(error ->
                errors.put(error.getField(), error.getDefaultMessage())
            );

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(errors);
    }
}

```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/exception/VehicleNotFoundException.java

```

package com.ridelink.driver_vehicle_service.exception;

public class VehicleNotFoundException extends RuntimeException {

    public VehicleNotFoundException(String id) {
        super("Vehicle not found with id: " + id);
    }
}

```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/model/Driver.java

```
package com.ridelink.driver_vehicle_service.model;

import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Document(collection = "drivers")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Driver {

    @Id
    private String id;

    private String accountId;

    private String licenseNumber;

    private String serviceArea;

    @Builder.Default
    private DriverStatus availabilityStatus = DriverStatus.OFFLINE;

    private Double latitude;

    private Double longitude;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/model/DriverStatus.java

```
package com.ridelink.driver_vehicle_service.model;

public enum DriverStatus {
    AVAILABLE,
    UNAVAILABLE,
    OFFLINE
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/model/Vehicle.java

```
package com.ridelink.driver_vehicle_service.model;

import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Document(collection = "vehicles")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Vehicle {

    @Id
    private String id;

    private String driverId;

    private String registrationNumber;

    private String brand;

    private String model;

    private VehicleType vehicleType;

    private String color;

    private Integer year;

    @Builder.Default
    private VehicleStatus status = VehicleStatus.ACTIVE;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/model/VehicleStatus.java

```
package com.ridelink.driver_vehicle_service.model;

public enum VehicleStatus {
    ACTIVE,
    INACTIVE
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/model/VehicleType.java

```
package com.ridelink.driver_vehicle_service.model;

public enum VehicleType {
    CAR,
    VAN,
    THREE_WHEELER
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/repository/DriverRepository.java

```
package com.ridelink.driver_vehicle_service.repository;

import com.ridelink.driver_vehicle_service.model.Driver;
import com.ridelink.driver_vehicle_service.model.DriverStatus;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.util.List;
import java.util.Optional;

public interface DriverRepository extends MongoRepository<Driver, String> {

    Optional<Driver> findByAccountId(String accountId);

    Optional<Driver> findByLicenseNumber(String licenseNumber);

    boolean existsByLicenseNumber(String licenseNumber);

    List<Driver> findByAvailabilityStatus(DriverStatus availabilityStatus);

    List<Driver> findByAvailabilityStatusAndServiceArea(
        DriverStatus availabilityStatus,
        String serviceArea
    );
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/repository/VehicleRepository.java

```
package com.ridelink.driver_vehicle_service.repository;

import com.ridelink.driver_vehicle_service.model.Vehicle;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.util.List;
import java.util.Optional;

public interface VehicleRepository extends MongoRepository<Vehicle, String> {

    List<Vehicle> findByDriverId(String driverId);

    Optional<Vehicle> findByRegistrationNumber(String registrationNumber);

    boolean existsByRegistrationNumber(String registrationNumber);
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/service/DriverService.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import java.util.List;

public interface DriverService {

    DriverResponse createDriver(CreateDriverRequest request);

    DriverResponse getDriverById(String id);

    DriverResponse updateDriver(String id, UpdateDriverRequest request);

    DriverResponse updateAvailability(String id, AvailabilityUpdateRequest request);

    DriverResponse updateLocation(String id, LocationUpdateRequest request);

    DriverResponse updateServiceArea(String id, ServiceAreaUpdateRequest request);

    List<DriverResponse> getAvailableDrivers();

    List<DriverResponse> getAvailableDriversByServiceArea(String serviceArea);

    void deleteDriver(String id);
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/service/DriverServiceImpl.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.exception.DriverNotFoundException;
import com.ridelink.driver_vehicle_service.exception.DuplicateLicenseException;
import com.ridelink.driver_vehicle_service.model.Driver;
import com.ridelink.driver_vehicle_service.model.DriverStatus;
import com.ridelink.driver_vehicle_service.repository.DriverRepository;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;

@Service
public class DriverServiceImpl implements DriverService {

    private final DriverRepository driverRepository;

    public DriverServiceImpl(DriverRepository driverRepository) {
        this.driverRepository = driverRepository;
    }

    @Override
    public DriverResponse createDriver(CreateDriverRequest request) {

        if (driverRepository.existsByLicenseNumber(request.getLicenseNumber())) {
            throw new DuplicateLicenseException(request.getLicenseNumber());
        }

        Driver driver = Driver.builder()
            .accountId(request.getAccountId())
            .licenseNumber(request.getLicenseNumber())
            .serviceArea(request.getServiceArea())
            .createdAt(LocalDateTime.now())
            .build();

        return mapToResponse(driverRepository.save(driver));
    }

    @Override
```

```
public DriverResponse getDriverById(String id) {  
    Driver driver = driverRepository.findById(id)  
        .orElseThrow(() -> new DriverNotFoundException(id));  
    return mapToResponse(driver);  
}  
  
@Override  
public DriverResponse updateDriver(String id, UpdateDriverRequest request) {  
    Driver driver = driverRepository.findById(id)  
        .orElseThrow(() -> new DriverNotFoundException(id));  
  
    if (request.getLicenseNumber() != null) {  
        driver.setLicenseNumber(request.getLicenseNumber());  
    }  
  
    if (request.getServiceArea() != null) {  
        driver.setServiceArea(request.getServiceArea());  
    }  
  
    driver.setUpdatedAt(LocalDateTime.now());  
  
    return mapToResponse(driverRepository.save(driver));  
}  
  
@Override  
public DriverResponse updateAvailability(String id, AvailabilityUpdateRequest request) {  
    Driver driver = driverRepository.findById(id)  
        .orElseThrow(() -> new DriverNotFoundException(id));  
  
    driver.setAvailabilityStatus(request.getAvailabilityStatus());  
    driver.setUpdatedAt(LocalDateTime.now());  
  
    return mapToResponse(driverRepository.save(driver));  
}  
  
@Override
```

```
public DriverResponse updateLocation(String id, LocationUpdateRequest request) {

    Driver driver = driverRepository.findById(id)
        .orElseThrow(() -> new DriverNotFoundException(id));

    driver.setLatitude(request.getLatitude());
    driver.setLongitude(request.getLongitude());
    driver.setUpdatedAt(LocalDateTime.now());

    return mapToResponse(driverRepository.save(driver));
}

@Override
public DriverResponse updateServiceArea(String id, ServiceAreaUpdateRequest request) {

    Driver driver = driverRepository.findById(id)
        .orElseThrow(() -> new DriverNotFoundException(id));

    driver.setServiceArea(request.getServiceArea());
    driver.setUpdatedAt(LocalDateTime.now());

    return mapToResponse(driverRepository.save(driver));
}

@Override
public List<DriverResponse> getAvailableDrivers() {

    return driverRepository
        .findByAvailabilityStatus(DriverStatus.AVAILABLE)
        .stream()
        .map(this::mapToResponse)
        .toList();
}

@Override
public List<DriverResponse> getAvailableDriversByServiceArea(String serviceArea) {

    return driverRepository
        .findByAvailabilityStatusAndServiceArea(DriverStatus.AVAILABLE, serviceArea)
        .stream()

        .map(this::mapToResponse)
        .toList();
}

@Override
public void deleteDriver(String id) {
    if (!driverRepository.existsById(id)) {
        throw new DriverNotFoundException(id);
    }
    driverRepository.deleteById(id);
}

private DriverResponse mapToResponse(Driver driver) {

    return DriverResponse.builder()
        .id(driver.getId())
        .accountId(driver.getAccountId())
        .licenseNumber(driver.getLicenseNumber())
        .serviceArea(driver.getServiceArea())
        .availabilityStatus(driver.getAvailabilityStatus())
        .latitude(driver.getLatitude())
        .longitude(driver.getLongitude())
        .createdAt(driver.getCreatedAt())
        .updatedAt(driver.getUpdatedAt())
        .build();
}
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/service/VehicleService.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import java.util.List;

public interface VehicleService {

    VehicleResponse createVehicle(CreateVehicleRequest request);

    VehicleResponse getVehicleById(String id);

    List<VehicleResponse> getVehiclesByDriverId(String driverId);

    VehicleResponse updateVehicle(String id, UpdateVehicleRequest request);

    VehicleResponse updateVehicleStatus(String id, VehicleStatusUpdateRequest request);

    void deleteVehicle(String id);
}
```

■ File: driver-vehicle-service/src/main/java/com/ridelink/driver_vehicle_service/service/VehicleServiceImpl.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.exception.DriverNotFoundException;
import com.ridelink.driver_vehicle_service.exception.DuplicateVehicleRegistrationException;
import com.ridelink.driver_vehicle_service.exception.VehicleNotFoundException;
import com.ridelink.driver_vehicle_service.model.Vehicle;
import com.ridelink.driver_vehicle_service.model.VehicleStatus;
import com.ridelink.driver_vehicle_service.repository.DriverRepository;
import com.ridelink.driver_vehicle_service.repository.VehicleRepository;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class VehicleServiceImpl implements VehicleService {

    private final VehicleRepository vehicleRepository;
    private final DriverRepository driverRepository;

    public VehicleServiceImpl(VehicleRepository vehicleRepository, DriverRepository driverRepository) {
        this.vehicleRepository = vehicleRepository;
        this.driverRepository = driverRepository;
    }

    @Override
    public VehicleResponse createVehicle(CreateVehicleRequest request) {
        if (!driverRepository.existsById(request.getDriverId())) {
            throw new DriverNotFoundException(request.getDriverId());
        }

        if (vehicleRepository.existsByRegistrationNumber(request.getRegistrationNumber())) {
            throw new DuplicateVehicleRegistrationException(request.getRegistrationNumber());
        }

        LocalDateTime now = LocalDateTime.now();
        Vehicle vehicle = Vehicle.builder()
            .driverId(request.getDriverId())
```

```
        .registrationNumber(request.getRegistrationNumber())
        .brand(request.getBrand())
        .model(request.getModel())
        .vehicleType(request.getVehicleType())
        .color(request.getColor())
        .year(request.getYear())
        .status(VehicleStatus.ACTIVE)
        .createdAt(now)
        .updatedAt(now)
        .build();

    Vehicle savedVehicle = vehicleRepository.save(vehicle);
    return mapToResponse(savedVehicle);
}

@Override
public VehicleResponse getVehicleById(String id) {
    Vehicle vehicle = vehicleRepository.findById(id)
        .orElseThrow(() -> new VehicleNotFoundException(id));
    return mapToResponse(vehicle);
}

@Override
public List<VehicleResponse> getVehiclesByDriverId(String driverId) {
    if (!driverRepository.existsById(driverId)) {
        throw new DriverNotFoundException(driverId);
    }

    return vehicleRepository.findByDriverId(driverId).stream()
        .map(this::mapToResponse)
        .collect(Collectors.toList());
}

@Override
public VehicleResponse updateVehicle(String id, UpdateVehicleRequest request) {
    Vehicle vehicle = vehicleRepository.findById(id)
        .orElseThrow(() -> new VehicleNotFoundException(id));

    if (request.getRegistrationNumber() != null && !request.getRegistrationNumber().isBlank()) {
        if (!request.getRegistrationNumber().equals(vehicle.getRegistrationNumber()) &&
```

```

        vehicleRepository.existsByRegistrationNumber(request.getRegistrationNumber())) {
            throw new DuplicateVehicleRegistrationException(request.getRegistrationNumber());
        }
        vehicle.setRegistrationNumber(request.getRegistrationNumber());
    }

    if (request.getBrand() != null && !request.getBrand().isBlank()) {
        vehicle.setBrand(request.getBrand());
    }

    if (request.getModel() != null && !request.getModel().isBlank()) {
        vehicle.setModel(request.getModel());
    }

    if (request.getVehicleType() != null) {
        vehicle.setVehicleType(request.getVehicleType());
    }

    if (request.getColor() != null && !request.getColor().isBlank()) {
        vehicle.setColor(request.getColor());
    }

    if (request.getYear() != null) {
        vehicle.setYear(request.getYear());
    }

    vehicle.setUpdatedAt(LocalDateTime.now());
    Vehicle updatedVehicle = vehicleRepository.save(vehicle);
    return mapToResponse(updatedVehicle);
}

```

```

@Override
public VehicleResponse updateVehicleStatus(String id, VehicleStatusUpdateRequest request) {
    Vehicle vehicle = vehicleRepository.findById(id)
        .orElseThrow(() -> new VehicleNotFoundException(id));

    vehicle.setStatus(request.getStatus());
    vehicle.setUpdatedAt(LocalDateTime.now());

    Vehicle updatedVehicle = vehicleRepository.save(vehicle);
}

```

```

        return mapToResponse(updatedVehicle);
    }
}

```

```

@Override
public void deleteVehicle(String id) {
    if (!vehicleRepository.existsById(id)) {
        throw new VehicleNotFoundException(id);
    }
    vehicleRepository.deleteById(id);
}
}

```

```

private VehicleResponse mapToResponse(Vehicle vehicle) {
    return VehicleResponse.builder()
        .id(vehicle.getId())
        .driverId(vehicle.getDriverId())
        .registrationNumber(vehicle.getRegistrationNumber())
        .brand(vehicle.getBrand())
        .model(vehicle.getModel())
        .vehicleType(vehicle.getVehicleType())
        .color(vehicle.getColor())
        .year(vehicle.getYear())
        .status(vehicle.getStatus())
        .createdAt(vehicle.getCreatedAt())
        .updatedAt(vehicle.getUpdatedAt())
        .build();
}
}
}

```

■ File: driver-vehicle-service/src/test/java/com/ridelink/driver_vehicle_service/DriverVehicleServiceApplicationTests.java

```
package com.ridelink.driver_vehicle_service;

import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@Disabled("Integration test requiring MongoDB")
@SpringBootTest
class DriverVehicleServiceApplicationTests {

    @Test
    @Disabled("Requires active MongoDB instance")
    void contextLoads() {
    }

}
```

■ File: driver-vehicle-service/src/test/java/com/ridelink/driver_vehicle_service/service/DriverServiceImplTest.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.exception.DriverNotFoundException;
import com.ridelink.driver_vehicle_service.exception.DuplicateLicenseException;
import com.ridelink.driver_vehicle_service.model.Driver;
import com.ridelink.driver_vehicle_service.model.DriverStatus;
import com.ridelink.driver_vehicle_service.repository.DriverRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class DriverServiceImplTest {

    @Mock
    private DriverRepository driverRepository;

    @InjectMocks
    private DriverServiceImpl driverService;

    private Driver sampleDriver;

    @BeforeEach
    void setUp() {
        sampleDriver = Driver.builder()
            .id("driver-1")
            .accountId("acc-1")
            .licenseNumber("LIC-100")
        .build();
    }

}
```

```
        .serviceArea("Colombo")
        .availabilityStatus(DriverStatus.AVAILABLE)
        .latitude(6.9271)
        .longitude(79.8612)
        .createdAt(LocalDateTime.now())
        .updatedAt(LocalDateTime.now())
        .build();
    }

    @Test
    void createDriver_success() {
        CreateDriverRequest request = new CreateDriverRequest();
        request.setAccountId("acc-1");
        request.setLicenseNumber("LIC-100");
        request.setServiceArea("Colombo");

        when(driverRepository.existsByLicenseNumber("LIC-100")).thenReturn(false);
        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        DriverResponse response = driverService.createDriver(request);

        assertNotNull(response);
        assertEquals("driver-1", response.getId());
        assertEquals("acc-1", response.getAccountId());
        assertEquals("LIC-100", response.getLicenseNumber());
        assertEquals("Colombo", response.getServiceArea());
        verify(driverRepository, times(1)).save(any(Driver.class));
    }

    @Test
    void createDriver_duplicateLicense() {
        CreateDriverRequest request = new CreateDriverRequest();
        request.setAccountId("acc-1");
        request.setLicenseNumber("LIC-100");
        request.setServiceArea("Colombo");

        when(driverRepository.existsByLicenseNumber("LIC-100")).thenReturn(true);

        assertThrows(DuplicateLicenseException.class, () -> driverService.createDriver(request));
        verify(driverRepository, never()).save(any(Driver.class));
    }
}
```

```
}

@Test
void getDriverById_success() {
    when(driverRepository.findById("driver-1")).thenReturn(Optional.of(sampleDriver));

    DriverResponse response = driverService.getDriverById("driver-1");

    assertNotNull(response);
    assertEquals("driver-1", response.getId());
    assertEquals("LIC-100", response.getLicenseNumber());
}

@Test
void getDriverById_notFound() {
    when(driverRepository.findById("invalid-id")).thenReturn(Optional.empty());

    assertThrows(DriverNotFoundException.class, () -> driverService.getDriverById("invalid-id"));
}

@Test
void updateAvailability_success() {
    AvailabilityUpdateRequest request = new AvailabilityUpdateRequest();
    request.setAvailabilityStatus(DriverStatus.UNAVAILABLE);

    when(driverRepository.findById("driver-1")).thenReturn(Optional.of(sampleDriver));
    when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

    DriverResponse response = driverService.updateAvailability("driver-1", request);

    assertNotNull(response);
    assertEquals(DriverStatus.UNAVAILABLE, sampleDriver.getAvailabilityStatus());
    verify(driverRepository, times(1)).save(sampleDriver);
}

@Test
void updateLocation_success() {
    LocationUpdateRequest request = new LocationUpdateRequest();
    request.setLatitude(7.2906);
    request.setLongitude(80.6337);
```

```
        when(driverRepository.findById("driver-1")).thenReturn(Optional.of(sampleDriver));
        when(driverRepository.save(any(Driver.class))).thenReturn(sampleDriver);

        DriverResponse response = driverService.updateLocation("driver-1", request);

        assertNotNull(response);
        assertEquals(7.2906, sampleDriver.getLatitude());
        assertEquals(80.6337, sampleDriver.getLongitude());
        verify(driverRepository, times(1)).save(sampleDriver);
    }

    @Test
    void getAvailableDrivers_success() {
        when(driverRepository.findByAvailabilityStatus(DriverStatus.AVAILABLE))
            .thenReturn(List.of(sampleDriver));

        List<DriverResponse> responses = driverService.getAvailableDrivers();

        assertNotNull(responses);
        assertEquals(1, responses.size());
        assertEquals("driver-1", responses.get(0).getId());
    }

    @Test
    void getAvailableDriversByServiceArea_success() {
        when(driverRepository.findByAvailabilityStatusAndServiceArea(DriverStatus.AVAILABLE, "Colombo"))
            .thenReturn(List.of(sampleDriver));

        List<DriverResponse> responses = driverService.getAvailableDriversByServiceArea("Colombo");

        assertNotNull(responses);
        assertEquals(1, responses.size());
        assertEquals("Colombo", responses.get(0).getServiceArea());
    }
}
```

■ File: driver-vehicle-service/src/test/java/com/ridelink/driver_vehicle_service/service/VehicleServiceImplTest.java

```
package com.ridelink.driver_vehicle_service.service;

import com.ridelink.driver_vehicle_service.dto.*;
import com.ridelink.driver_vehicle_service.exception.DriverNotFoundException;
import com.ridelink.driver_vehicle_service.exception.DuplicateVehicleRegistrationException;
import com.ridelink.driver_vehicle_service.exception.VehicleNotFoundException;
import com.ridelink.driver_vehicle_service.model.Vehicle;
import com.ridelink.driver_vehicle_service.model.VehicleStatus;
import com.ridelink.driver_vehicle_service.model.VehicleType;
import com.ridelink.driver_vehicle_service.repository.DriverRepository;
import com.ridelink.driver_vehicle_service.repository.VehicleRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class VehicleServiceImplTest {

    @Mock
    private VehicleRepository vehicleRepository;

    @Mock
    private DriverRepository driverRepository;

    @InjectMocks
    private VehicleServiceImpl vehicleService;

    private Vehicle sampleVehicle;
```

```
@BeforeEach
void setUp() {
    sampleVehicle = Vehicle.builder()
        .id("vehicle-1")
        .driverId("driver-1")
        .registrationNumber("WP CAB-1234")
        .brand("Toyota")
        .model("Prius")
        .vehicleType(VehicleType.CAR)
        .color("White")
        .year(2022)
        .status(VehicleStatus.ACTIVE)
        .createdAt(LocalDateTime.now())
        .updatedAt(LocalDateTime.now())
        .build();
}

@Test
void createVehicle_success() {
    CreateVehicleRequest request = new CreateVehicleRequest();
    request.setDriverId("driver-1");
    request.setRegistrationNumber("WP CAB-1234");
    request.setBrand("Toyota");
    request.setModel("Prius");
    request.setVehicleType(VehicleType.CAR);
    request.setColor("White");
    request.setYear(2022);

    when(driverRepository.existsById("driver-1")).thenReturn(true);
    when(vehicleRepository.existsByRegistrationNumber("WP CAB-1234")).thenReturn(false);
    when(vehicleRepository.save(any(Vehicle.class))).thenReturn(sampleVehicle);

    VehicleResponse response = vehicleService.createVehicle(request);

    assertNotNull(response);
    assertEquals("vehicle-1", response.getId());
    assertEquals("driver-1", response.getDriverId());
    assertEquals("WP CAB-1234", response.getRegistrationNumber());
    assertEquals(VehicleStatus.ACTIVE, response.getStatus());
    verify(vehicleRepository, times(1)).save(any(Vehicle.class));
}
```

```
}

@Test
void createVehicle_duplicateRegistration() {
    CreateVehicleRequest request = new CreateVehicleRequest();
    request.setDriverId("driver-1");
    request.setRegistrationNumber("WP CAB-1234");

    when(driverRepository.existsById("driver-1")).thenReturn(true);
    when(vehicleRepository.existsByRegistrationNumber("WP CAB-1234")).thenReturn(true);

    assertThrows(DuplicateVehicleRegistrationException.class, () -> vehicleService.createVehicle(request));
    verify(vehicleRepository, never()).save(any(Vehicle.class));
}

@Test
void createVehicle_driverNotFound() {
    CreateVehicleRequest request = new CreateVehicleRequest();
    request.setDriverId("invalid-driver");
    request.setRegistrationNumber("WP CAB-1234");

    when(driverRepository.existsById("invalid-driver")).thenReturn(false);

    assertThrows(DriverNotFoundException.class, () -> vehicleService.createVehicle(request));
    verify(vehicleRepository, never()).save(any(Vehicle.class));
}

@Test
void getVehicleById_success() {
    when(vehicleRepository.findById("vehicle-1")).thenReturn(Optional.of(sampleVehicle));

    VehicleResponse response = vehicleService.getVehicleById("vehicle-1");

    assertNotNull(response);
    assertEquals("vehicle-1", response.getId());
    assertEquals("WP CAB-1234", response.getRegistrationNumber());
}

@Test
void getVehicleById_notFound() {
```

```
        when(vehicleRepository.findById("invalid-vehicle")).thenReturn(Optional.empty());

        assertThrows(VehicleNotFoundException.class, () -> vehicleService.getVehicleById("invalid-vehicle"));
    }
}
```

@Test

```
void updateVehicle_success() {
    UpdateVehicleRequest request = new UpdateVehicleRequest();
    request.setBrand("Honda");
    request.setModel("Civic");

    when(vehicleRepository.findById("vehicle-1")).thenReturn(Optional.of(sampleVehicle));
    when(vehicleRepository.save(any(Vehicle.class))).thenReturn(sampleVehicle);

    VehicleResponse response = vehicleService.updateVehicle("vehicle-1", request);

    assertNotNull(response);
    assertEquals("Honda", sampleVehicle.getBrand());
    assertEquals("Civic", sampleVehicle.getModel());
    verify(vehicleRepository, times(1)).save(sampleVehicle);
}
}
```

@Test

```
void updateVehicleStatus_success() {
    VehicleStatusUpdateRequest request = new VehicleStatusUpdateRequest();
    request.setStatus(VehicleStatus.INACTIVE);

    when(vehicleRepository.findById("vehicle-1")).thenReturn(Optional.of(sampleVehicle));
    when(vehicleRepository.save(any(Vehicle.class))).thenReturn(sampleVehicle);

    VehicleResponse response = vehicleService.updateVehicleStatus("vehicle-1", request);

    assertNotNull(response);
    assertEquals(VehicleStatus.INACTIVE, sampleVehicle.getStatus());
    verify(vehicleRepository, times(1)).save(sampleVehicle);
}
}
```

@Test

```
void getVehiclesByDriverId_success() {
    when(driverRepository.existsById("driver-1")).thenReturn(true);

    when(vehicleRepository.findByDriverId("driver-1")).thenReturn(List.of(sampleVehicle));

    List<VehicleResponse> responses = vehicleService.getVehiclesByDriverId("driver-1");

    assertNotNull(responses);
    assertEquals(1, responses.size());
    assertEquals("vehicle-1", responses.get(0).getId());
}
}
```

3. Ride Management Service Module

Assigned Lead: Perera H.G.K.D (Student ID: IT24104143)

Overview: Handles ride request creation, passenger and driver tracking, location pickup/dropoff points, ride status state machine (REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED).

■ File: ride-management-service/.mvn\wrapper\maven-wrapper.properties

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.16/apache-maven-3.9.16-bin.zip
```

■ File: ride-management-service/src/main/resources/application.properties

```
spring.application.name=ride-management-service

spring.mongodb.uri=${MONGODB_URI}
spring.mongodb.database=ridelink_ride_management

server.port=8081
```

■ File: ride-management-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>

  <groupId>com.ridelink</groupId>
  <artifactId>ride-management-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <name>ride-management-service</name>
  <description>Ride Management Service for RideLink</description>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>

    <!-- Spring Web -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <!-- MongoDB -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>

    <!-- Validation -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Swagger / OpenAPI -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.0</version>
</dependency>

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- DevTools -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>

<!-- Tests -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb-test</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation-test</artifactId>
  <scope>test</scope>
</dependency>
```

```

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc-test</artifactId>
      <scope>test</scope>
    </dependency>

  </dependencies>

  <build>
    <plugins>

      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>

        <executions>

          <execution>
            <id>default-compile</id>
            <phase>compile</phase>
            <goals>
              <goal>compile</goal>
            </goals>

            <configuration>
              <annotationProcessorPaths>
                <path>
                  <groupId>org.projectlombok</groupId>
                  <artifactId>lombok</artifactId>
                </path>
              </annotationProcessorPaths>
            </configuration>
          </execution>

          <execution>
            <id>default-testCompile</id>
            <phase>test-compile</phase>
            <goals>
              <goal>testCompile</goal>
            </goals>

            <configuration>
              <annotationProcessorPaths>
                <path>
                  <groupId>org.projectlombok</groupId>
                  <artifactId>lombok</artifactId>
                </path>
              </annotationProcessorPaths>
            </configuration>
          </execution>

        </executions>
      </plugin>

    </plugins>
  </build>
</project>

```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/RideManagementServiceApplication.java

```
package com.ridelink.ride_management_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RideManagementServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(RideManagementServiceApplication.class, args);
    }

}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/controller/RideController.java

```
package com.ridelink.ride_management_service.controller;

import com.ridelink.ride_management_service.model.Ride;
import com.ridelink.ride_management_service.model.RideStatus;
import com.ridelink.ride_management_service.service.RideService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/rides")
@RequiredArgsConstructor
@Tag(name = "Ride Management", description = "APIs for managing rides and ride status")
public class RideController {

    private final RideService rideService;

    @PostMapping
    @Operation(summary = "Create a new ride")
    @ApiResponse(responseCode = "201", description = "Successful operation")
    @ApiResponse(responseCode = "400", description = "Invalid input")
    public ResponseEntity<Ride> createRide(@Valid @RequestBody Ride ride) {
        Ride createdRide = rideService.createRide(ride);
        return ResponseEntity.status(HttpStatus.CREATED).body(createdRide);
    }

    @GetMapping
    @Operation(summary = "Get all rides")
    @ApiResponse(responseCode = "200", description = "Successful operation")
    public ResponseEntity<List<Ride>> getAllRides() {
        return ResponseEntity.ok(rideService.getAllRides());
    }

}
```

```
@GetMapping("/{id}")
@Operation(summary = "Get ride by ID")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Successful operation"),
    @ApiResponse(responseCode = "404", description = "Ride not found")
})
public ResponseEntity<Ride> getRideById(@PathVariable String id) {
    Optional<Ride> rideOptional = rideService.getRideById(id);
    return rideOptional.map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}

@PutMapping("/{id}")
@Operation(summary = "Update ride details")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Successful operation"),
    @ApiResponse(responseCode = "400", description = "Invalid input"),
    @ApiResponse(responseCode = "404", description = "Ride not found")
})
public ResponseEntity<Ride> updateRide(@PathVariable String id, @Valid @RequestBody Ride ride) {
    Ride updatedRide = rideService.updateRide(id, ride);
    if (updatedRide == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(updatedRide);
}

@PatchMapping("/{id}/status")
@Operation(summary = "Update ride status")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "Successful operation"),
    @ApiResponse(responseCode = "400", description = "Invalid input"),
    @ApiResponse(responseCode = "404", description = "Ride not found")
})
public ResponseEntity<Ride> updateRideStatus(@PathVariable String id, @RequestParam RideStatus status) {
    Ride updatedRide = rideService.updateRideStatus(id, status);
    if (updatedRide == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(updatedRide);
}

@DeleteMapping("/{id}")
@Operation(summary = "Delete a ride")
@ApiResponses({
    @ApiResponse(responseCode = "204", description = "Successful operation"),
    @ApiResponse(responseCode = "404", description = "Ride not found")
})
public ResponseEntity<Void> deleteRide(@PathVariable String id) {
    boolean deleted = rideService.deleteRide(id);
    if (deleted) {
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/exception/ApiError.java

```
package com.ridelink.ride_management_service.exception;

import lombok.AllArgsConstructor;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@AllArgsConstructor
public class ApiError {
    private int status;
    private String error;
    private String message;
    private LocalDateTime timestamp;
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.ride_management_service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.LocalDateTime;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleMethodArgumentNotValid(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult().getFieldErrors().stream()
            .map(fieldError -> fieldError.getField() + ": " + fieldError.getDefaultMessage())
            .collect(Collectors.joining("; "));
        ApiError apiError = new ApiError(
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            errors,
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiError> handleMethodArgumentTypeMismatch(MethodArgumentTypeMismatchException ex) {
        String message = String.format("Invalid value '%s' for parameter '%s'", ex.getValue(), ex.getName());
        ApiError apiError = new ApiError(
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            message,
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
    }
}
```

```
@ExceptionHandler(IllegalStateException.class)
public ResponseEntity<ApiError> handleIllegalState(IllegalStateException ex) {
    ApiError apiError = new ApiError(
        HttpStatus.BAD_REQUEST.value(),
        HttpStatus.BAD_REQUEST.getReasonPhrase(),
        ex.getMessage(),
        LocalDateTime.now()
    );
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/model/Ride.java

```
package com.ridelink.ride_management_service.model;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Document(collection = "rides")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Ride {

    @Id
    private String id;

    @NotBlank(message = "Passenger ID must not be blank")
    private String passengerId;

    private String driverId;

    @NotBlank(message = "Pickup location must not be blank")
    private String pickupLocation;

    @NotBlank(message = "Dropoff location must not be blank")
    private String dropoffLocation;

    private RideStatus status;

    @NotNull(message = "Fare must not be null")
    private Double fare;

    private LocalDateTime requestedAt;
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/model/RideStatus.java

```
package com.ridelink.ride_management_service.model;

public enum RideStatus {
    REQUESTED,
    ACCEPTED,
    STARTED,
    COMPLETED,
    CANCELLED
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/repository/RideRepository.java

```
package com.ridelink.ride_management_service.repository;

import com.ridelink.ride_management_service.model.Ride;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface RideRepository extends MongoRepository<Ride, String> {
}
```

■ File: ride-management-service/src/main/java/com/ridelink/ride_management_service/elservice/RideService.java

```
package com.ridelink.ride_management_service.service;

import com.ridelink.ride_management_service.model.Ride;
import com.ridelink.ride_management_service.model.RideStatus;
import com.ridelink.ride_management_service.repository.RideRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Service
@RequiredArgsConstructor
public class RideService {

    private final RideRepository rideRepository;

    public Ride createRide(Ride ride) {
        ride.setStatus(RideStatus.REQUESTED);
        ride.setRequestedAt(LocalDateTime.now());
        return rideRepository.save(ride);
    }

    public List<Ride> getAllRides() {
        return rideRepository.findAll();
    }

    public Optional<Ride> getRideById(String id) {
        return rideRepository.findById(id);
    }

    public Ride updateRide(String id, Ride updatedRide) {
        Optional<Ride> existingRideOptional = rideRepository.findById(id);
        if (existingRideOptional.isPresent()) {
            Ride existingRide = existingRideOptional.get();
            existingRide.setPassengerId(updatedRide.getPassengerId());
            existingRide.setDriverId(updatedRide.getDriverId());
            existingRide.setPickupLocation(updatedRide.getPickupLocation());
            existingRide.setDropoffLocation(updatedRide.getDropoffLocation());
        }
    }
}
```

```

        existingRide.setFare(updatedRide.getFare());
        return rideRepository.save(existingRide);
    }
    return null;
}

public Ride updateRideStatus(String id, RideStatus newStatus) {
    Optional<Ride> existingRideOptional = rideRepository.findById(id);
    if (existingRideOptional.isEmpty()) {
        return null;
    }

    Ride existingRide = existingRideOptional.get();
    RideStatus currentStatus = existingRide.getStatus();

    if (!isValidTransition(currentStatus, newStatus)) {
        throw new IllegalStateException("Invalid status transition from " + currentStatus + " to " + newStatus);
    }

    existingRide.setStatus(newStatus);
    return rideRepository.save(existingRide);
}

private boolean isValidTransition(RideStatus currentStatus, RideStatus newStatus) {
    if (currentStatus == null || newStatus == null) {
        return false;
    }
    return switch (currentStatus) {
        case REQUESTED -> newStatus == RideStatus.ACCEPTED || newStatus == RideStatus.CANCELLED;
        case ACCEPTED -> newStatus == RideStatus.STARTED || newStatus == RideStatus.CANCELLED;
        case STARTED -> newStatus == RideStatus.COMPLETED;
        default -> false;
    };
}

public boolean deleteRide(String id) {
    if (rideRepository.existsById(id)) {
        rideRepository.deleteById(id);
        return true;
    }

    return false;
}
}

```

■ File: ride-management-service/src/test/java/com/ridelink/ride_management_service/RideManagementServiceApplicationTests.java

```

package com.ridelink.ride_management_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class RideManagementServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}

```

4. Account Service Module

Assigned Lead: Bandara J.K.T.I (Student ID: IT24104276)

Overview: Provides core user identity management, user creation, credentials verification, profile updates, and role management (PASSENGER, DRIVER, ADMIN).

■ File: account-service/.mvn\wrapper\maven-wrapper.properties

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.16/apache-maven-3.9.16-bin.zip
```

■ File: account-service/src/main/resources\application.properties

```
spring.application.name=account-service

spring.mongodb.uri=${MONGODB_URI}
spring.mongodb.database=ridelink_account

server.port=8080

jwt.secret=${JWT_SECRET}
jwt.expiration=3600000
```

■ File: account-service/src/test/resources\application.properties

```
spring.mongodb.uri=mongodb://localhost:27017/ridelink_account_test
spring.mongodb.database=ridelink_account_test

jwt.secret=VGhpc0lzQVN1ZmZpY2llbnRseUxvbmdUZXR0U2VjcWV0S2V5MTIzNA==
jwt.expiration=3600000
```

■ File: account-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>

  <groupId>com.ridelink</groupId>
  <artifactId>account-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <name>account-service</name>
  <description>Account Service for RideLink</description>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>

    <!-- Spring Web MVC -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <!-- MongoDB -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>

    <!-- Validation -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Swagger / OpenAPI -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.0</version>
</dependency>

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- DevTools -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>

<!-- MongoDB Test -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb-test</artifactId>
  <scope>test</scope>
</dependency>

<!-- Web MVC Test -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
```

```
<!-- Validation Test -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation-test</artifactId>
  <scope>test</scope>
</dependency>

<!-- Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
</dependencies>

<build>
  <plugins>

    <plugin>
      <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>

    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>

        <executions>

            <execution>
                <id>default-compile</id>
                <phase>compile</phase>
                <goals>
                    <goal>compile</goal>
                </goals>

                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.project.lombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </execution>

            <execution>
                <id>default-testCompile</id>
                <phase>test-compile</phase>
                <goals>
                    <goal>testCompile</goal>
                </goals>

                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.project.lombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </execution>

        </executions>

    </plugin>

</plugins>
</build>

</project>
```

■ File: account-service/src/main/java/com/ridelink/account_service/AccountServiceApplication.java

```
package com.ridelink.account_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccountServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(AccountServiceApplication.class, args);
    }

}
```

■ File: account-service/src/main/java/com/ridelink/account_service/config/PasswordEncoderConfig.java

```
package com.ridelink.account_service.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

■ File: account-service/src/main/java/com/ridelink/account_service/config/SecurityConfig.java

```
package com.ridelink.account_service.config;

import com.ridelink.account_service.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                    "/api/accounts/register",
                    "/api/accounts/login",
                    "/swagger-ui/**",
                    "/swagger-ui.html",
                    "/v3/api-docs/**"
                ).permitAll()
                .requestMatchers(HttpMethod.GET, "/api/accounts/{id}").hasAnyRole("PASSENGER", "DRIVER", "ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/accounts/{id}/profile").hasAnyRole("PASSENGER", "DRIVER", "ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/accounts").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/accounts/{id}/status").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/accounts/{id}/role").hasRole("ADMIN")

                .requestMatchers(HttpMethod.DELETE, "/api/accounts/{id}").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/controller/AccountController.java

```
package com.ridelink.account_service.controller;

import com.ridelink.account_service.dto.AccountResponse;
import com.ridelink.account_service.dto.LoginRequest;
import com.ridelink.account_service.dto.LoginResponse;
import com.ridelink.account_service.dto.RegisterRequest;
import com.ridelink.account_service.dto.UpdateProfileRequest;
import com.ridelink.account_service.dto.UpdateRoleRequest;
import com.ridelink.account_service.dto.UpdateStatusRequest;
import com.ridelink.account_service.model.Account;
import com.ridelink.account_service.model.AccountRole;
import com.ridelink.account_service.service.AccountService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/accounts")
@Tag(
    name = "Account Management",
    description = "APIs for account registration, login, profile management, role management and account status management"
)
public class AccountController {

    private final AccountService accountService;

    public AccountController(AccountService accountService) {
        this.accountService = accountService;
    }

    @PostMapping("/register")
    @Operation(summary = "Register a passenger or driver account")
    @ApiResponse(responseCode = "201", description = "Created")
```

```
@ApiResponse(responseCode = "400", description = "Bad Request")
public ResponseEntity<AccountResponse> registerAccount(@Valid @RequestBody RegisterRequest request) {
    return new ResponseEntity<>(accountService.registerAccount(request), HttpStatus.CREATED);
}

@PostMapping("/login")
@Operation(summary = "Login and receive JWT token")
@ApiResponse(responseCode = "200", description = "OK")
@ApiResponse(responseCode = "400", description = "Bad Request")
public ResponseEntity<LoginResponse> login(@Valid @RequestBody LoginRequest request) {
    return ResponseEntity.ok(accountService.login(request));
}

@GetMapping("/{id}")
@Operation(summary = "Get account profile by ID")
@ApiResponse(responseCode = "200", description = "OK")
@ApiResponse(responseCode = "404", description = "Not Found")
public ResponseEntity<AccountResponse> getAccountById(@PathVariable String id, Authentication authentication) {
    Account authenticatedAccount = (Account) authentication.getPrincipal();
    if (authenticatedAccount.getRole() != AccountRole.ADMIN && !authenticatedAccount.getId().equals(id)) {
        throw new IllegalStateException("Access denied");
    }
    return ResponseEntity.ok(accountService.getAccountById(id));
}

@GetMapping
@Operation(summary = "Get all accounts")
@ApiResponse(responseCode = "200", description = "OK")
public ResponseEntity<List<AccountResponse>> getAllAccounts() {
    return ResponseEntity.ok(accountService.getAllAccounts());
}

@PutMapping("/{id}/profile")
@Operation(summary = "Update account profile")
@ApiResponse(responseCode = "200", description = "OK")
@ApiResponse(responseCode = "400", description = "Bad Request")
@ApiResponse(responseCode = "404", description = "Not Found")
public ResponseEntity<AccountResponse> updateProfile(
    @PathVariable String id,
    @Valid @RequestBody UpdateProfileRequest request,
```

```

        Authentication authentication) {
    Account authenticatedAccount = (Account) authentication.getPrincipal();
    if (authenticatedAccount.getRole() != AccountRole.ADMIN && !authenticatedAccount.getId().equals(id)) {
        throw new IllegalStateException("Access denied");
    }
    return ResponseEntity.ok(accountService.updateProfile(id, request));
}

@PatchMapping("/{id}/status")
@Operation(summary = "Update account status")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "OK"),
    @ApiResponse(responseCode = "400", description = "Bad Request"),
    @ApiResponse(responseCode = "404", description = "Not Found")
})
public ResponseEntity<AccountResponse> updateAccountStatus(
    @PathVariable String id,
    @Valid @RequestBody UpdateStatusRequest request) {
    return ResponseEntity.ok(accountService.updateAccountStatus(id, request));
}

@PatchMapping("/{id}/role")
@Operation(summary = "Update account role")
@ApiResponses({
    @ApiResponse(responseCode = "200", description = "OK"),
    @ApiResponse(responseCode = "400", description = "Bad Request"),
    @ApiResponse(responseCode = "404", description = "Not Found")
})
public ResponseEntity<AccountResponse> updateAccountRole(
    @PathVariable String id,
    @Valid @RequestBody UpdateRoleRequest request) {
    return ResponseEntity.ok(accountService.updateAccountRole(id, request));
}

@DeleteMapping("/{id}")
@Operation(summary = "Delete an account")
@ApiResponses({
    @ApiResponse(responseCode = "204", description = "No Content"),
    @ApiResponse(responseCode = "404", description = "Not Found")
})
public ResponseEntity<Void> deleteAccount(@PathVariable String id) {
    accountService.deleteAccount(id);
    return ResponseEntity.noContent().build();
}
}

```

■ File: account-service/src/main/java/com/ridelink/account_service/dto/AccountResponse.java

```

package com.ridelink.account_service.dto;

import com.ridelink.account_service.model.AccountRole;
import com.ridelink.account_service.model.AccountStatus;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class AccountResponse {
    private String id;
    private String fullName;
    private String email;
    private String phoneNumber;
    private AccountRole role;
    private AccountStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

```

■ File:**account-service/src/main/java/com/ridelink/account_service/dto/LoginRequest.java**

```
package com.ridelink.account_service.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class LoginRequest {
    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String password;
}
```

■ File:**account-service/src/main/java/com/ridelink/account_service/dto/LoginResponse.java**

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.model.AccountRole;
import com.ridelink.account_service.model.AccountStatus;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class LoginResponse {
    private String token;
    private String tokenType;
    private String accountId;
    private String email;
    private AccountRole role;
    private AccountStatus status;
}
```

■ File:**account-service/src/main/java/com/ridelink/account_service/dto/RegisterRequest.java**

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.model.AccountRole;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class RegisterRequest {
    @NotBlank
    private String fullName;

    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String phoneNumber;

    @NotBlank
    private String password;

    @NotNull
    private AccountRole role;
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateProfile Request.java

```
package com.ridelink.account_service.dto;

import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UpdateProfileRequest {
    @NotBlank
    private String fullName;

    @NotBlank
    private String phoneNumber;
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateRoleRequest.java

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.model.AccountRole;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UpdateRoleRequest {
    @NotNull
    private AccountRole role;
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/dto/UpdateStatusRequest.java

```
package com.ridelink.account_service.dto;

import com.ridelink.account_service.model.AccountStatus;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class UpdateStatusRequest {
    @NotNull
    private AccountStatus status;
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/exception/ApiError.java

```
package com.ridelink.account_service.exception;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class ApiError {
    private int status;
    private String error;
    private String message;
    private LocalDateTime timestamp;
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.account_service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<ApiError> handleIllegalStateException(IllegalStateException ex) {
        String message = ex.getMessage();
        HttpStatus status = HttpStatus.BAD_REQUEST;

        if ("Account not found".equals(message)) {
            status = HttpStatus.NOT_FOUND;
        } else if ("Invalid email or password".equals(message)) {
            status = HttpStatus.UNAUTHORIZED;
        } else if ("Account is not active".equals(message) || "Access denied".equals(message)) {
            status = HttpStatus.FORBIDDEN;
        } else if ("Email already registered".equals(message) || "Invalid registration role".equals(message)) {
            status = HttpStatus.BAD_REQUEST;
        }

        ApiError apiError = new ApiError(
            status.value(),
            status.getReasonPhrase(),
            message,
            LocalDateTime.now()
        );
        return new ResponseEntity<>(apiError, status);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
```

```
public ResponseEntity<ApiError> handleValidationException(MethodArgumentNotValidException ex) {
    String errorMessage = "Validation failed";
    if (ex.getBindingResult().hasErrors()) {
        errorMessage = ex.getBindingResult().getAllErrors().get(0).getDefaultMessage();
    }

    HttpStatus status = HttpStatus.BAD_REQUEST;
    ApiError apiError = new ApiError(
        status.value(),
        status.getReasonPhrase(),
        errorMessage,
        LocalDateTime.now()
    );
    return new ResponseEntity<>(apiError, status);
}

@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ApiError> handleTypeMismatchException(MethodArgumentTypeMismatchException ex) {
    HttpStatus status = HttpStatus.BAD_REQUEST;
    ApiError apiError = new ApiError(
        status.value(),
        status.getReasonPhrase(),
        "Invalid request parameter",
        LocalDateTime.now()
    );
    return new ResponseEntity<>(apiError, status);
}

@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ApiError> handleHttpMessageNotReadableException(HttpMessageNotReadableException ex) {
    HttpStatus status = HttpStatus.BAD_REQUEST;
    ApiError apiError = new ApiError(
        status.value(),
        status.getReasonPhrase(),
        "Invalid request body",
        LocalDateTime.now()
    );
    return new ResponseEntity<>(apiError, status);
}
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/mapper/AccountMapper.java

```
package com.ridelink.account_service.mapper;

import com.ridelink.account_service.dto.AccountResponse;
import com.ridelink.account_service.dto.RegisterRequest;
import com.ridelink.account_service.model.Account;
import org.springframework.stereotype.Component;

@Component
public class AccountMapper {

    public Account toEntity(RegisterRequest request) {
        if (request == null) {
            return null;
        }
        Account account = new Account();
        account.setFullName(request.getFullName());
        account.setEmail(request.getEmail());
        account.setPhoneNumber(request.getPhoneNumber());
        account.setPassword(request.getPassword());
        account.setRole(request.getRole());
        return account;
    }

    public AccountResponse toResponse(Account account) {
        if (account == null) {
            return null;
        }
        AccountResponse response = new AccountResponse();
        response.setId(account.getId());
        response.setFullName(account.getFullName());
        response.setEmail(account.getEmail());
        response.setPhoneNumber(account.getPhoneNumber());
        response.setRole(account.getRole());
        response.setStatus(account.getStatus());
        response.setCreatedAt(account.getCreatedAt());
        response.setUpdatedAt(account.getUpdatedAt());
        return response;
    }
}
```

■ File:**account-service/src/main/java/com/ridelink/account_service/model/Account.java**

```
package com.ridelink.account_service.model;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "accounts")
public class Account {

    @Id
    private String id;

    @NotBlank
    private String fullName;

    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String phoneNumber;

    @NotBlank
    private String password;

    private AccountRole role;

    private AccountStatus status;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;
}
```

■ File:**account-service/src/main/java/com/ridelink/account_service/model/AccountRole.java**

```
package com.ridelink.account_service.model;

public enum AccountRole {
    PASSENGER,
    DRIVER,
    ADMIN
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/model/AccountStatus.java

```
package com.ridelink.account_service.model;

public enum AccountStatus {
    ACTIVE,
    INACTIVE,
    SUSPENDED
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/repository/AccountRepository.java

```
package com.ridelink.account_service.repository;

import com.ridelink.account_service.model.Account;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface AccountRepository extends MongoRepository<Account, String> {
    Optional<Account> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/security/JwtAuthenticationFilter.java

```
package com.ridelink.account_service.security;

import com.ridelink.account_service.model.Account;
import com.ridelink.account_service.model.AccountStatus;
import com.ridelink.account_service.repository.AccountRepository;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Collections;
import java.util.Optional;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final AccountRepository accountRepository;

    public JwtAuthenticationFilter(JwtService jwtService, AccountRepository accountRepository) {
        this.jwtService = jwtService;
        this.accountRepository = accountRepository;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {
        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```
final String jwt = authHeader.substring(7);

try {
    if (jwtService.isTokenValid(jwt) && SecurityContextHolder.getContext().getAuthentication() == null) {
        String email = jwtService.extractEmail(jwt);
        Optional<Account> accountOpt = accountRepository.findByEmail(email);

        if (accountOpt.isPresent()) {
            Account account = accountOpt.get();
            if (account.getStatus() == AccountStatus.ACTIVE) {
                String role = account.getRole().name();
                SimpleGrantedAuthority authority = new SimpleGrantedAuthority("ROLE_" + role);

                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                    account,
                    null,
                    Collections.singletonList(authority)
                );
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }
    } catch (Exception e) {
        // Ignore parsing/validation exceptions and continue the chain without authenticating
    }

    filterChain.doFilter(request, response);
}
}
```

■ File:**account-service/src/main/java/com/ridelink/account_service/security/JwtService.java**

```
package com.ridelink.account_service.security;

import com.ridelink.account_service.model.Account;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.security.Key;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private long jwtExpiration;

    public String generateToken(Account account) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("accountId", account.getId());
        claims.put("role", account.getRole().name());

        return Jwts.builder()
            .setClaims(claims)
            .setSubject(account.getEmail())
            .setIssuedAt(new Date(System.currentTimeMillis()))
            .setExpiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSignInKey(), SignatureAlgorithm.HS256)
            .compact();
    }
}
```

```
public String extractEmail(String token) {
    return extractAllClaims(token).getSubject();
}

public String extractRole(String token) {
    return extractAllClaims(token).get("role", String.class);
}

public String extractAccountId(String token) {
    return extractAllClaims(token).get("accountId", String.class);
}

public boolean isTokenValid(String token) {
    try {
        Claims claims = extractAllClaims(token);
        return !claims.getExpiration().before(new Date());
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

private Claims extractAllClaims(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(getSignInKey())
        .build()
        .parseClaimsJws(token)
        .getBody();
}

private Key getSignInKey() {
    byte[] keyBytes = Decoders.BASE64.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}
}
```

■ File: account-service/src/main/java/com/ridelink/account_service/service/AccountService.java

```
package com.ridelink.account_service.service;

import com.ridelink.account_service.dto.AccountResponse;
import com.ridelink.account_service.dto.LoginRequest;
import com.ridelink.account_service.dto.LoginResponse;
import com.ridelink.account_service.dto.RegisterRequest;
import com.ridelink.account_service.dto.UpdateProfileRequest;
import com.ridelink.account_service.dto.UpdateRoleRequest;
import com.ridelink.account_service.dto.UpdateStatusRequest;
import com.ridelink.account_service.mapper.AccountMapper;
import com.ridelink.account_service.model.Account;
import com.ridelink.account_service.model.AccountRole;
import com.ridelink.account_service.model.AccountStatus;
import com.ridelink.account_service.repository.AccountRepository;
import com.ridelink.account_service.security.JwtService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class AccountService {

    private final AccountRepository accountRepository;
    private final AccountMapper accountMapper;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public AccountService(AccountRepository accountRepository, AccountMapper accountMapper, PasswordEncoder passwordEncoder) {
        this.accountRepository = accountRepository;
        this.accountMapper = accountMapper;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    public AccountResponse registerAccount(RegisterRequest request) {
        if (request.getRole() != AccountRole.PASSENGER && request.getRole() != AccountRole.DRIVER) {
            throw new IllegalStateException("Invalid registration role");
        }
    }
}
```

```
    }

    if (accountRepository.existsByEmail(request.getEmail())) {
        throw new IllegalStateException("Email already registered");
    }

    Account account = accountMapper.toEntity(request);
    account.setPassword(passwordEncoder.encode(request.getPassword()));
    account.setStatus(AccountStatus.ACTIVE);
    account.setCreatedAt(LocalDateTime.now());
    account.setUpdatedAt(LocalDateTime.now());

    Account savedAccount = accountRepository.save(account);
    return accountMapper.toResponse(savedAccount);
}

public LoginResponse login(LoginRequest request) {
    Account account = accountRepository.findByEmail(request.getEmail())
        .orElseThrow(() -> new IllegalStateException("Invalid email or password"));

    if (!passwordEncoder.matches(request.getPassword(), account.getPassword())) {
        throw new IllegalStateException("Invalid email or password");
    }

    if (account.getStatus() != AccountStatus.ACTIVE) {
        throw new IllegalStateException("Account is not active");
    }

    String token = jwtService.generateToken(account);
    return new LoginResponse(
        token,
        "Bearer",
        account.getId(),
        account.getEmail(),
        account.getRole(),
        account.getStatus()
    );
}

public AccountResponse getAccountById(String id) {
```

```
        return accountMapper.toResponse(findAccountEntityById(id));
    }

    public AccountResponse getAccountByEmail(String email) {
        Account account = accountRepository.findByEmail(email)
            .orElseThrow(() -> new IllegalStateException("Account not found"));
        return accountMapper.toResponse(account);
    }

    public List<AccountResponse> getAllAccounts() {
        return accountRepository.findAll().stream()
            .map(accountMapper::toResponse)
            .collect(Collectors.toList());
    }

    public AccountResponse updateProfile(String id, UpdateProfileRequest request) {
        Account existing = findAccountEntityById(id);
        existing.setFullName(request.getFullName());
        existing.setPhoneNumber(request.getPhoneNumber());
        existing.setUpdatedAt(LocalDateTime.now());

        Account updatedAccount = accountRepository.save(existing);
        return accountMapper.toResponse(updatedAccount);
    }

    public AccountResponse updateAccountStatus(String id, UpdateStatusRequest request) {
        Account existing = findAccountEntityById(id);
        existing.setStatus(request.getStatus());
        existing.setUpdatedAt(LocalDateTime.now());

        Account updatedAccount = accountRepository.save(existing);
        return accountMapper.toResponse(updatedAccount);
    }

    public AccountResponse updateAccountRole(String id, UpdateRoleRequest request) {
        Account existing = findAccountEntityById(id);
        existing.setRole(request.getRole());
        existing.setUpdatedAt(LocalDateTime.now());

        Account updatedAccount = accountRepository.save(existing);

        return accountMapper.toResponse(updatedAccount);
    }

    public void deleteAccount(String id) {
        if (!accountRepository.existsById(id)) {
            throw new IllegalStateException("Account not found");
        }
        accountRepository.deleteById(id);
    }

    private Account findAccountEntityById(String id) {
        return accountRepository.findById(id)
            .orElseThrow(() -> new IllegalStateException("Account not found"));
    }
}
```

■ File: account-service/src/test/java/com/ridelink/account_service/AccountServiceApplicationTests.java

```
package com.ridelink.account_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class AccountServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

■ File: account-service/src/test/java/com/ridelink/account_service/service/AccountServiceTest.java

```
package com.ridelink.account_service.service;

import com.ridelink.account_service.dto.*;
import com.ridelink.account_service.mapper.AccountMapper;
import com.ridelink.account_service.model.Account;
import com.ridelink.account_service.model.AccountRole;
import com.ridelink.account_service.model.AccountStatus;
import com.ridelink.account_service.repository.AccountRepository;
import com.ridelink.account_service.security.JwtService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.time.LocalDateTime;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class AccountServiceTest {

    @Mock
    private AccountRepository accountRepository;

    @Mock
    private AccountMapper accountMapper;

    @Mock
    private PasswordEncoder passwordEncoder;

    @Mock
    private JwtService jwtService;
```

```
@InjectMocks
private AccountService accountService;

private Account mockAccount;
private RegisterRequest mockRegisterRequest;
private LoginRequest mockLoginRequest;

@BeforeEach
void setUp() {
    mockAccount = new Account();
    mockAccount.setId("acc-123");
    mockAccount.setFullName("John Doe");
    mockAccount.setEmail("john@example.com");
    mockAccount.setPassword("hashed-password");
    mockAccount.setRole(AccountRole.PASSENGER);
    mockAccount.setStatus(AccountStatus.ACTIVE);
    mockAccount.setCreatedAt(LocalDateTime.now());
    mockAccount.setUpdatedAt(LocalDateTime.now());

    mockRegisterRequest = new RegisterRequest();
    mockRegisterRequest.setFullName("John Doe");
    mockRegisterRequest.setEmail("john@example.com");
    mockRegisterRequest.setPassword("raw-password");
    mockRegisterRequest.setPhoneNumber("1234567890");
    mockRegisterRequest.setRole(AccountRole.PASSENGER);

    mockLoginRequest = new LoginRequest();
    mockLoginRequest.setEmail("john@example.com");
    mockLoginRequest.setPassword("raw-password");
}

@Test
void registerAccount_success() {
    when(accountRepository.existsByEmail(anyString())).thenReturn(false);
    when(accountMapper.toEntity(any(RegisterRequest.class))).thenReturn(mockAccount);
    when(passwordEncoder.encode(anyString())).thenReturn("hashed-password");
    when(accountRepository.save(any(Account.class))).thenReturn(mockAccount);

    AccountResponse mockResponse = new AccountResponse();
    mockResponse.setId("acc-123");
```

```
        when(accountMapper.toResponse(any(Account.class))).thenReturn(mockResponse);

        AccountResponse response = accountService.registerAccount(mockRegisterRequest);

        assertNotNull(response);
        assertEquals("acc-123", response.getId());
        verify(accountRepository).save(any(Account.class));
        verify(passwordEncoder).encode("raw-password");
    }

    @Test
    void registerAccount_duplicateEmail() {
        when(accountRepository.existsByEmail(anyString())).thenReturn(true);

        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.registerAccount(mockRegisterRequest);
        });

        assertEquals("Email already registered", exception.getMessage());
        verify(accountRepository, never()).save(any(Account.class));
    }

    @Test
    void registerAccount_adminRoleRejected() {
        mockRegisterRequest.setRole(AccountRole.ADMIN);

        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.registerAccount(mockRegisterRequest);
        });

        assertEquals("Invalid registration role", exception.getMessage());
        verify(accountRepository, never()).save(any(Account.class));
    }

    @Test
    void login_success() {
        when(accountRepository.findByEmail(anyString())).thenReturn(Optional.of(mockAccount));
        when(passwordEncoder.matches(anyString(), anyString())).thenReturn(true);
        when(jwtService.generateToken(any(Account.class))).thenReturn("mock-jwt-token");
    }
```

```
        LoginResponse response = accountService.login(mockLoginRequest);

        assertNotNull(response);
        assertEquals("mock-jwt-token", response.getToken());
        assertEquals("Bearer", response.getTokenType());
        assertEquals("acc-123", response.getAccountId());
        assertEquals("john@example.com", response.getEmail());
        assertEquals(AccountRole.PASSENGER, response.getRole());
        assertEquals(AccountStatus.ACTIVE, response.getStatus());
    }

    @Test
    void login_invalidPassword() {
        when(accountRepository.findByEmail(anyString())).thenReturn(Optional.of(mockAccount));
        when(passwordEncoder.matches(anyString(), anyString())).thenReturn(false);

        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.login(mockLoginRequest);
        });

        assertEquals("Invalid email or password", exception.getMessage());
    }

    @Test
    void login_accountNotFound() {
        when(accountRepository.findByEmail(anyString())).thenReturn(Optional.empty());

        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.login(mockLoginRequest);
        });

        assertEquals("Invalid email or password", exception.getMessage());
    }

    @Test
    void login_inactiveAccount() {
        mockAccount.setStatus(AccountStatus.INACTIVE);
        when(accountRepository.findByEmail(anyString())).thenReturn(Optional.of(mockAccount));
        when(passwordEncoder.matches(anyString(), anyString())).thenReturn(true);
    }
```

```
        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.login(mockLoginRequest);
        });

        assertEquals("Account is not active", exception.getMessage());
    }

    @Test
    void getAccountById_success() {
        when(accountRepository.findById("acc-123")).thenReturn(Optional.of(mockAccount));
        when(accountMapper.toResponse(mockAccount)).thenReturn(new AccountResponse());

        AccountResponse response = accountService.getAccountById("acc-123");

        assertNotNull(response);
    }

    @Test
    void getAccountById_notFound() {
        when(accountRepository.findById("acc-999")).thenReturn(Optional.empty());

        IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
            accountService.getAccountById("acc-999");
        });

        assertEquals("Account not found", exception.getMessage());
    }

    @Test
    void updateProfile_success() {
        UpdateProfileRequest request = new UpdateProfileRequest();
        request.setFullName("Jane Doe");
        request.setPhoneNumber("0987654321");

        when(accountRepository.findById("acc-123")).thenReturn(Optional.of(mockAccount));
        when(accountRepository.save(any(Account.class))).thenReturn(mockAccount);
        when(accountMapper.toResponse(any(Account.class))).thenReturn(new AccountResponse());

        AccountResponse response = accountService.updateProfile("acc-123", request);
    }
```

```
        assertNotNull(response);
        assertEquals("Jane Doe", mockAccount.getFullName());
        assertEquals("0987654321", mockAccount.getPhoneNumber());
        assertEquals("john@example.com", mockAccount.getEmail());
        assertEquals(AccountRole.PASSENGER, mockAccount.getRole());
        assertEquals(AccountStatus.ACTIVE, mockAccount.getStatus());
        verify(accountRepository).save(mockAccount);
    }

    @Test
    void updateAccountStatus_success() {
        UpdateStatusRequest request = new UpdateStatusRequest();
        request.setStatus(AccountStatus.SUSPENDED);

        when(accountRepository.findById("acc-123")).thenReturn(Optional.of(mockAccount));
        when(accountRepository.save(any(Account.class))).thenReturn(mockAccount);
        when(accountMapper.toResponse(any(Account.class))).thenReturn(new AccountResponse());

        AccountResponse response = accountService.updateAccountStatus("acc-123", request);

        assertNotNull(response);
        assertEquals(AccountStatus.SUSPENDED, mockAccount.getStatus());
        verify(accountRepository).save(mockAccount);
    }

    @Test
    void updateAccountRole_success() {
        UpdateRoleRequest request = new UpdateRoleRequest();
        request.setRole(AccountRole.DRIVER);

        when(accountRepository.findById("acc-123")).thenReturn(Optional.of(mockAccount));
        when(accountRepository.save(any(Account.class))).thenReturn(mockAccount);
        when(accountMapper.toResponse(any(Account.class))).thenReturn(new AccountResponse());

        AccountResponse response = accountService.updateAccountRole("acc-123", request);

        assertNotNull(response);
        assertEquals(AccountRole.DRIVER, mockAccount.getRole());
        verify(accountRepository).save(mockAccount);
    }
}
```

```
@Test
void deleteAccount_success() {
    when(accountRepository.existsById("acc-123")).thenReturn(true);

    assertDoesNotThrow(() -> {
        accountService.deleteAccount("acc-123");
    });

    verify(accountRepository).deleteById("acc-123");
}

@Test
void deleteAccount_notFound() {
    when(accountRepository.existsById("acc-999")).thenReturn(false);

    IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
        accountService.deleteAccount("acc-999");
    });

    assertEquals("Account not found", exception.getMessage());
    verify(accountRepository, never()).deleteById(anyString());
}
}
```

5. Fare & Payment Service Module

Assigned Lead: Karunarathna H.W.D.S.S (Student ID: IT24104353)

Overview: Calculates estimated and final ride fares based on distance and duration metrics, generates fare receipts, and processes payment transactions (CASH, CARD).

■ File: fare-payment-service/.mvn\wrapper\maven-wrapper.properties

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.16/apache-maven-3.9.16-bin.zip
```

■ File: fare-payment-service/src/main/resources/application.properties

```
spring.application.name=fare-payment-service
```

■ File: fare-payment-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>

  <groupId>com.ridelink</groupId>
  <artifactId>fare-payment-service</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <name>fare-payment-service</name>
  <description>RideLink Fare and Payment Microservice</description>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>

    <!-- Spring Web / REST API -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <!-- MongoDB -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>

    <!-- Validation -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- OpenAPI / Swagger UI -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.0.0</version>
</dependency>

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- DevTools -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>

    <plugin>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>

<configuration>
  <excludes>
    <exclude>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
    </exclude>
  </excludes>
</configuration>
</plugin>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>

  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>

</plugins>
</build>

</project>
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/FarePaymentServiceApplication.java

```
package com.ridelink.fare_payment_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FarePaymentServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(FarePaymentServiceApplication.class, args);
    }

}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/config/OpenApiConfig.java

```
package com.ridelink.fare_payment_service.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("RideLink Fare & Payment Service API")
                .description("REST API for fare estimation, fare finalization, and simulated payments.")
                .version("v1"));
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/controller/FareController.java

```
package com.ridelink.fare_payment_service.controller;

import com.ridelink.fare_payment_service.dto.FareEstimateRequest;
import com.ridelink.fare_payment_service.dto.FareResponse;
import com.ridelink.fare_payment_service.dto.FinalFareRequest;
import com.ridelink.fare_payment_service.service.FareService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/fares")
public class FareController {

    private final FareService fareService;

    public FareController(FareService fareService) {
        this.fareService = fareService;
    }

    @PostMapping("/estimate")
    public ResponseEntity<FareResponse> estimateFare(@Valid @RequestBody FareEstimateRequest request) {
        FareResponse response = fareService.estimateFare(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/{fareId}/finalize")
    public ResponseEntity<FareResponse> finalizeFare(@PathVariable String fareId,
        @Valid @RequestBody FinalFareRequest request) {
        FareResponse response = fareService.finalizeFare(fareId, request);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{fareId}")
    public ResponseEntity<FareResponse> getFareById(@PathVariable String fareId) {
        FareResponse response = fareService.getFareById(fareId);
        return ResponseEntity.ok(response);
    }
}
```

```
@GetMapping("/ride/{rideId}")
public ResponseEntity<FareResponse> getFareByRideId(@PathVariable String rideId) {
    FareResponse response = fareService.getFareByRideId(rideId);
    return ResponseEntity.ok(response);
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/controller/PaymentController.java

```
package com.ridelink.fare_payment_service.controller;

import com.ridelink.fare_payment_service.dto.CreatePaymentRequest;
import com.ridelink.fare_payment_service.dto.PaymentResponse;
import com.ridelink.fare_payment_service.service.PaymentService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/payments")
public class PaymentController {

    private final PaymentService paymentService;

    public PaymentController(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    @PostMapping
    public ResponseEntity<PaymentResponse> createPayment(@Valid @RequestBody CreatePaymentRequest request) {
        PaymentResponse response = paymentService.createPayment(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/{paymentId}")
    public ResponseEntity<PaymentResponse> getPaymentById(@PathVariable String paymentId) {
        PaymentResponse response = paymentService.getPaymentById(paymentId);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/ride/{rideId}")
    public ResponseEntity<PaymentResponse> getPaymentByRideId(@PathVariable String rideId) {
        PaymentResponse response = paymentService.getPaymentByRideId(rideId);
        return ResponseEntity.ok(response);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/dto/CreatePaymentRequest.java

```
package com.ridelink.fare_payment_service.dto;

import com.ridelink.fare_payment_service.model.PaymentMethod;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CreatePaymentRequest {

    @NotBlank(message = "Ride ID is required")
    private String rideId;

    @NotBlank(message = "Fare ID is required")
    private String fareId;

    @NotNull(message = "Payment method is required")
    private PaymentMethod paymentMethod;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/dto/FareEstimateRequest.java

```
package com.ridelink.fare_payment_service.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class FareEstimateRequest {

    @NotBlank(message = "Ride ID is required")
    private String rideId;

    @NotNull(message = "Distance is required")
    @DecimalMin(value = "0.0", message = "Distance must be greater than or equal to 0")
    private Double distanceKm;

    @NotNull(message = "Duration is required")
    @DecimalMin(value = "0.0", message = "Duration must be greater than or equal to 0")
    private Double durationMinutes;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/dto/FareResponse.java

```
package com.ridelink.fare_payment_service.dto;

import com.ridelink.fare_payment_service.model.FareStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class FareResponse {

    private String id;
    private String rideId;
    private Double distanceKm;
    private Double durationMinutes;
    private Double baseFare;
    private Double distanceFare;
    private Double timeFare;
    private Double estimatedFare;
    private Double finalFare;
    private FareStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/dto/FinalFareRequest.java

```
package com.ridelink.fare_payment_service.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class FinalFareRequest {

    @NotNull(message = "Distance is required")
    @DecimalMin(value = "0.0", message = "Distance must be greater than or equal to 0")
    private Double distanceKm;

    @NotNull(message = "Duration is required")
    @DecimalMin(value = "0.0", message = "Duration must be greater than or equal to 0")
    private Double durationMinutes;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/dto/PaymentResponse.java

```
package com.ridelink.fare_payment_service.dto;

import com.ridelink.fare_payment_service.model.PaymentMethod;
import com.ridelink.fare_payment_service.model.PaymentStatus;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class PaymentResponse {

    private String id;
    private String rideId;
    private String fareId;
    private Double amount;
    private PaymentMethod paymentMethod;
    private PaymentStatus paymentStatus;
    private String transactionReference;
    private LocalDateTime createdAt;
    private LocalDateTime paidAt;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/DuplicateFareException.java

```
package com.ridelink.fare_payment_service.exception;

public class DuplicateFareException extends RuntimeException {
    public DuplicateFareException(String message) {
        super(message);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/DuplicatePaymentException.java

```
package com.ridelink.fare_payment_service.exception;

public class DuplicatePaymentException extends RuntimeException {
    public DuplicatePaymentException(String message) {
        super(message);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/FareNotFinalizedException.java

```
package com.ridelink.fare_payment_service.exception;

public class FareNotFinalizedException extends RuntimeException {
    public FareNotFinalizedException(String message) {
        super(message);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/FareNotFoundException.java

```
package com.ridelink.fare_payment_service.exception;

public class FareNotFoundException extends RuntimeException {
    public FareNotFoundException(String message) {
        super(message);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/GlobalExceptionHandler.java

```
package com.ridelink.fare_payment_service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler({FareNotFoundException.class, PaymentNotFoundException.class})
    public ResponseEntity<Map<String, Object>> handleNotFoundException(RuntimeException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, "Not Found", ex.getMessage());
    }

    @ExceptionHandler({DuplicateFareException.class, DuplicatePaymentException.class})
    public ResponseEntity<Map<String, Object>> handleConflictException(RuntimeException ex) {
        return buildResponse(HttpStatus.CONFLICT, "Conflict", ex.getMessage());
    }

    @ExceptionHandler({FareNotFinalizedException.class, IllegalArgumentException.class})
    public ResponseEntity<Map<String, Object>> handleBadRequestException(RuntimeException ex) {
        return buildResponse(HttpStatus.BAD_REQUEST, "Bad Request", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationException(MethodArgumentNotValidException ex) {
        String errorMessage = ex.getBindingResult().getFieldErrors().stream()
            .map(error -> error.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return buildResponse(HttpStatus.BAD_REQUEST, "Bad Request", errorMessage);
    }

    private ResponseEntity<Map<String, Object>> buildResponse(HttpStatus status, String error, String message) {

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", status.value());
        body.put("error", error);
        body.put("message", message);
        return new ResponseEntity<>(body, status);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/exception/PaymentNotFoundException.java

```
package com.ridelink.fare_payment_service.exception;

public class PaymentNotFoundException extends RuntimeException {
    public PaymentNotFoundException(String message) {
        super(message);
    }
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/model/Fare.java

```
package com.ridelink.fare_payment_service.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Document(collection = "fares")
public class Fare {

    @Id
    private String id;
    private String rideId;
    private Double distanceKm;
    private Double durationMinutes;
    private Double baseFare;
    private Double distanceFare;
    private Double timeFare;
    private Double estimatedFare;
    private Double finalFare;
    private FareStatus status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/model/FareStatus.java

```
package com.ridelink.fare_payment_service.model;

public enum FareStatus {
    ESTIMATED,
    FINALIZED
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/model/Payment.java

```
package com.ridelink.fare_payment_service.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.LocalDateTime;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Document(collection = "payments")
public class Payment {

    @Id
    private String id;
    private String rideId;
    private String fareId;
    private Double amount;
    private PaymentMethod paymentMethod;
    private PaymentStatus paymentStatus;
    private String transactionReference;
    private LocalDateTime createdAt;
    private LocalDateTime paidAt;
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/model/PaymentMethod.java

```
package com.ridelink.fare_payment_service.model;

public enum PaymentMethod {
    CASH,
    CARD
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/model/PaymentStatus.java

```
package com.ridelink.fare_payment_service.model;

public enum PaymentStatus {
    PENDING,
    PAID,
    FAILED
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/repository/FareRepository.java

```
package com.ridelink.fare_payment_service.repository;

import com.ridelink.fare_payment_service.model.Fare;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface FareRepository extends MongoRepository<Fare, String> {

    Optional<Fare> findById(String rideId);

    boolean existsByRideId(String rideId);
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/repository/PaymentRepository.java

```
package com.ridelink.fare_payment_service.repository;

import com.ridelink.fare_payment_service.model.Payment;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface PaymentRepository extends MongoRepository<Payment, String> {

    Optional<Payment> findById(String rideId);

    Optional<Payment> findById(String fareId);

    boolean existsByFareId(String fareId);
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/service/FareService.java

```
package com.ridelink.fare_payment_service.service;

import com.ridelink.fare_payment_service.dto.FareEstimateRequest;
import com.ridelink.fare_payment_service.dto.FareResponse;
import com.ridelink.fare_payment_service.dto.FinalFareRequest;

public interface FareService {

    FareResponse estimateFare(FareEstimateRequest request);

    FareResponse finalizeFare(String fareId, FinalFareRequest request);

    FareResponse getFareById(String fareId);

    FareResponse getFareByRideId(String rideId);
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/service/IService.java

```
package com.ridelink.fare_payment_service.service;

import com.ridelink.fare_payment_service.dto.CreatePaymentRequest;
import com.ridelink.fare_payment_service.dto.PaymentResponse;

public interface PaymentService {

    PaymentResponse createPayment(CreatePaymentRequest request);

    PaymentResponse getPaymentById(String paymentId);

    PaymentResponse getPaymentByRideId(String rideId);

}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/service/impl/FareServiceImpl.java

```
package com.ridelink.fare_payment_service.service.impl;

import com.ridelink.fare_payment_service.dto.FareEstimateRequest;
import com.ridelink.fare_payment_service.dto.FareResponse;
import com.ridelink.fare_payment_service.dto.FinalFareRequest;
import com.ridelink.fare_payment_service.exception.DuplicateFareException;
import com.ridelink.fare_payment_service.exception.FareNotFoundException;
import com.ridelink.fare_payment_service.model.Fare;
import com.ridelink.fare_payment_service.model.FareStatus;
import com.ridelink.fare_payment_service.repository.FareRepository;
import com.ridelink.fare_payment_service.service.FareService;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;

@Service
public class FareServiceImpl implements FareService {

    private static final double BASE_FARE = 200.00;
    private static final double DISTANCE_RATE = 100.00; // per km
    private static final double TIME_RATE = 20.00; // per minute

    private final FareRepository fareRepository;

    public FareServiceImpl(FareRepository fareRepository) {
        this.fareRepository = fareRepository;
    }

    @Override
    public FareResponse estimateFare(FareEstimateRequest request) {
        if (fareRepository.existsByRideId(request.getRideId())) {
            throw new DuplicateFareException("Fare already estimated for ride ID: " + request.getRideId());
        }

        double distanceFare = request.getDistanceKm() * DISTANCE_RATE;
        double timeFare = request.getDurationMinutes() * TIME_RATE;
        double estimatedFare = BASE_FARE + distanceFare + timeFare;

        LocalDateTime now = LocalDateTime.now();
```

```
Fare fare = Fare.builder()
    .rideId(request.getRideId())
    .distanceKm(request.getDistanceKm())
    .durationMinutes(request.getDurationMinutes())
    .baseFare(BASE_FARE)
    .distanceFare(distanceFare)
    .timeFare(timeFare)
    .estimatedFare(estimatedFare)
    .finalFare(null)
    .status(FareStatus.ESTIMATED)
    .createdAt(now)
    .updatedAt(now)
    .build();

Fare savedFare = fareRepository.save(fare);
return mapToFareResponse(savedFare);
}

@Override
public FareResponse finalizeFare(String fareId, FinalFareRequest request) {
    Fare fare = fareRepository.findById(fareId)
        .orElseThrow(() -> new FareNotFoundException("Fare not found with ID: " + fareId));

    double distanceFare = request.getDistanceKm() * DISTANCE_RATE;
    double timeFare = request.getDurationMinutes() * TIME_RATE;
    double finalFare = BASE_FARE + distanceFare + timeFare;

    fare.setDistanceKm(request.getDistanceKm());
    fare.setDurationMinutes(request.getDurationMinutes());
    fare.setDistanceFare(distanceFare);
    fare.setTimeFare(timeFare);
    fare.setFinalFare(finalFare);
    fare.setStatus(FareStatus.FINALIZED);
    fare.setUpdatedAt(LocalDate.now());

    Fare updatedFare = fareRepository.save(fare);
    return mapToFareResponse(updatedFare);
}

@Override
```

```
public FareResponse getFareById(String fareId) {
    Fare fare = fareRepository.findById(fareId)
        .orElseThrow(() -> new FareNotFoundException("Fare not found with ID: " + fareId));
    return mapToFareResponse(fare);
}

@Override
public FareResponse getFareByRideId(String rideId) {
    Fare fare = fareRepository.findById(rideId)
        .orElseThrow(() -> new FareNotFoundException("Fare not found for ride ID: " + rideId));
    return mapToFareResponse(fare);
}

private FareResponse mapToFareResponse(Fare fare) {
    return FareResponse.builder()
        .id(fare.getId())
        .rideId(fare.getRideId())
        .distanceKm(fare.getDistanceKm())
        .durationMinutes(fare.getDurationMinutes())
        .baseFare(fare.getBaseFare())
        .distanceFare(fare.getDistanceFare())
        .timeFare(fare.getTimeFare())
        .estimatedFare(fare.getEstimatedFare())
        .finalFare(fare.getFinalFare())
        .status(fare.getStatus())
        .createdAt(fare.getCreatedAt())
        .updatedAt(fare.getUpdatedAt())
        .build();
}
}
```

■ File: fare-payment-service/src/main/java/com/ridelink/fare_payment_service/service/impl/PaymentServiceImpl.java

```
package com.ridelink.fare_payment_service.service.impl;

import com.ridelink.fare_payment_service.dto.CreatePaymentRequest;
import com.ridelink.fare_payment_service.dto.PaymentResponse;
import com.ridelink.fare_payment_service.exception.DuplicatePaymentException;
import com.ridelink.fare_payment_service.exception.FareNotFoundException;
import com.ridelink.fare_payment_service.exception.FareNotFinalizedException;
import com.ridelink.fare_payment_service.exception.PaymentNotFoundException;
import com.ridelink.fare_payment_service.model.Fare;
import com.ridelink.fare_payment_service.model.FareStatus;
import com.ridelink.fare_payment_service.model.Payment;
import com.ridelink.fare_payment_service.model.PaymentStatus;
import com.ridelink.fare_payment_service.repository.FareRepository;
import com.ridelink.fare_payment_service.repository.PaymentRepository;
import com.ridelink.fare_payment_service.service.PaymentService;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.UUID;

@Service
public class PaymentServiceImpl implements PaymentService {

    private final PaymentRepository paymentRepository;
    private final FareRepository fareRepository;

    public PaymentServiceImpl(PaymentRepository paymentRepository, FareRepository fareRepository) {
        this.paymentRepository = paymentRepository;
        this.fareRepository = fareRepository;
    }

    @Override
    public PaymentResponse createPayment(CreatePaymentRequest request) {
        Fare fare = fareRepository.findById(request.getFareId())
            .orElseThrow(() -> new FareNotFoundException("Fare not found with ID: " + request.getFareId()));

        if (!fare.getRideId().equals(request.getRideId())) {
            throw new IllegalArgumentException("Fare does not belong to ride ID: " + request.getRideId());
        }
    }
}
```

```
        if (fare.getStatus() != FareStatus.FINALIZED) {
            throw new FareNotFinalizedException("Payment can only be processed for a FINALIZED fare");
        }

        if (paymentRepository.existsByFareId(request.getFareId())) {
            throw new DuplicatePaymentException("Payment already created for fare ID: " + request.getFareId());
        }

        LocalDateTime now = LocalDateTime.now();
        String transactionRef = "TXN-" + UUID.randomUUID().toString().toUpperCase();

        Payment payment = Payment.builder()
            .rideId(request.getRideId())
            .fareId(request.getFareId())
            .amount(fare.getFinalFare())
            .paymentMethod(request.getPaymentMethod())
            .paymentStatus(PaymentStatus.PAID)
            .transactionReference(transactionRef)
            .createdAt(now)
            .paidAt(now)
            .build();

        Payment savedPayment = paymentRepository.save(payment);
        return mapToPaymentResponse(savedPayment);
    }

    @Override
    public PaymentResponse getPaymentById(String paymentId) {
        Payment payment = paymentRepository.findById(paymentId)
            .orElseThrow(() -> new PaymentNotFoundException("Payment not found with ID: " + paymentId));
        return mapToPaymentResponse(payment);
    }

    @Override
    public PaymentResponse getPaymentByRideId(String rideId) {
        Payment payment = paymentRepository.findByRideId(rideId)
            .orElseThrow(() -> new PaymentNotFoundException("Payment not found for ride ID: " + rideId));
        return mapToPaymentResponse(payment);
    }

    private PaymentResponse mapToPaymentResponse(Payment payment) {
        return PaymentResponse.builder()
            .id(payment.getId())
            .rideId(payment.getRideId())
            .fareId(payment.getFareId())
            .amount(payment.getAmount())
            .paymentMethod(payment.getPaymentMethod())
            .paymentStatus(payment.getPaymentStatus())
            .transactionReference(payment.getTransactionReference())
            .createdAt(payment.getCreatedAt())
            .paidAt(payment.getPaidAt())
            .build();
    }
}
```

■ File: fare-payment-service/src\test\java\com\ridelink\fare_payment_service\FarePaymentServiceApplicationTests.java

```
package com.ridelink.fare_payment_service;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class FarePaymentServiceApplicationTests {

    @Test
    void contextLoads() {
    }

}
```